Low-Level Design (LLD) - Pulse Rates List Tracker
(Python Core & List Operations)

Difficulty Level: Medium | Total Marks: 20

Design Format: 1 Class | 7 Methods | 7 Test Cases

(Marks allocation details in "Test Cases & Marks Allocation" section)

Summary of Design Requirements

Implement a class 'HealthMonitor' to track and analyze patient heart rates using nested list operations. The class focuses on data cleaning, identify peak health readings, and calculating medical trends.

The class should:

- Load patient health logs from a CSV file.
- Handle missing heart rate values by cleaning the dataset.
- Provide statistical insights such as averages and extreme values.
- Monitor risk levels by identifying and counting high-risk patients.

Concepts Tested

- List and nested lists state management and schemas
- Data cleaning (None handling in lists)
- Aggregation and Grouping (manual averages, max)
- Value filtering and extraction
- Unique value extraction and sorting

Problem Statement

Design a health monitoring tool 'HealthMonitor' to analyze heart rate data. The system starts by loading 'health.csv' containing patient vitals. The monitor must be capable of cleaning incomplete records, identifying peak readings, and generating reports on patient averages and risk categories.

Note: The foundational methods for initialization and data loading (Method 1 & 2) are pre-provided.

Operations (Methods)

1. Initialize Monitor [0 Marks]

def __init__(self):

- Logic: Initialize 'self.records = None'.

2. Read Heart Data [0 Marks]

def read_data(self, file_path: str):

- Logic: Load the CSV file into the self.records list of lists where each nested list represents a row: [PatientID (str), Day (str), HeartRate (int or None)].

3. Clean Records [4 Marks]

def clean_records(self) -> int:

- Logic: Drop all records where 'HeartRate' is None. Update the internal list in place.

- Return: The number of records removed.

4. Find Highest Rate [4 Marks]

def find_highest_rate(self) -> int:

- Logic: Find the maximum heart rate value across all cleaned records.
- Return: The peak integer heart rate.

5. Average Heart Rate per Patient [4 Marks]

def patient_averages(self) -> dict:

- Logic: Calculate mean 'HeartRate' for each 'PatientID' (rounded to 2 decimal places).
- Return: Dictionary with 'PatientID' as keys and average heart rates as values.

6. High Risk Patients [4 Marks]

def high_risk(self, threshold: int) -> list:

- Logic: Identify unique 'PatientID' values who have at least one reading above 'threshold'.
- Return: Sorted list of unique high-risk Patient IDs.

7. Count High Risk [4 Marks]

def count_high_risk(self, threshold: int) -> int:

- Logic: Count how many unique patients meet the high-risk criteria (> 'threshold').
- Return: Integer count of high-risk patients.

Test Cases & Marks Allocation

Test Case ID	Description	Method	Marks
TC1	Verify initial state (Sample)	__init__	0
TC2	Verify data loading (Sample)	read_data()	0
TC3	Verify removal of missing values	clean_records()	4
TC4	Identify highest recorded rate	find_highest_rate()	4
TC5	Calculate health averages per patient	patient_averages()	4
TC6	Identify patients exceeding 100 BPM	high_risk()	4
TC7	Count unique high-risk individuals	count_high_risk()	4

Visible Test Case Descriptions

- TC1: self.records is None upon class creation.
- TC2: read_data() loads the sample data into self.records (12 rows initially).
- TC3: clean_records() returns 2 (Removing records for P02 and P06).
- TC4: find_highest_rate() returns 110.
- TC5: patient_averages()['P01'] returns approximately 72.33.
- TC6: high_risk(100) returns ['P03'].
- TC7: count_high_risk(100) returns 1.